Technical Requirements Document (TRD)

Portfolio Website – Shib Chandan Mistry

---

1. System Overview

1.1 Purpose

This document defines the technical architecture, components, APIs, data flow, and implementation details required to build the portfolio website.

1.2 System Type

- Static + dynamic hybrid frontend application
- Server-side rendering (SSR) for performance and SEO

---

2. High-Level Architecture

2.1 Architecture Type

- Monolithic frontend (Next.js-based)
- Static content-driven (no backend required initially)

2.2 Architecture Flow

User → Browser → Next.js App → Static Data (JSON/MDX)

Optional future:
User → Frontend → API → Database

---

3. Tech Stack

3.1 Frontend

- Framework: Next.js (App Router)
- Language: JavaScript / TypeScript (recommended)
- Styling: Tailwind CSS
- Animations: Framer Motion

3.2 Deployment

- Hosting: Vercel
- CDN: Edge caching via Vercel

3.3 Tooling

- Version Control: Git + GitHub
- Package Manager: npm
- Build Tool: Next.js default (Webpack/Turbopack)

---

4. Application Structure

4.1 Folder Structure

```
/app
  /components
    Navbar.jsx
    Hero.jsx
    ProjectCard.jsx
    ProjectModal.jsx
    Footer.jsx

  /data
    projects.js
    skills.js

  /sections
    About.jsx
    Projects.jsx
    Skills.jsx
    Contact.jsx
```

/styles

globals.css

page.jsx

---

5. Data Design

5.1 Project Data Schema

```
{
  id: string,
  title: string,
  shortDescription: string,
  fullDescription: string,
  techStack: string[],
  githubLink: string,
  demoLink: string,
  category: string,
  features: string[],
  challenges: string[],
  learnings: string[]
}
```

5.2 Skills Schema

```
{
  category: string,
  items: string[]
```

}

---

6. Component Design

6.1 Navbar

- Sticky navigation
- Scroll-based highlighting
- Mobile responsive menu

6.2 Hero Section

- Name + tagline
- CTA buttons:
    - View Projects
    - Download Resume

6.3 Project Card

Functional Requirements:

- Display:
    - Title
    - Short description
    - Tech stack
- Buttons:
    - GitHub
    - Demo

Interaction:

- Hover animation
- Click → open modal

6.4 Project Modal (Deep Dive)

Contains:

- Full description
- Architecture explanation
- Feature list
- Challenges
- Learnings

6.5 Skills Section

- Categorized grid layout
- Responsive design

6.6 Contact Section

- Email display
- External links (GitHub, LinkedIn)
- Optional form (future)

---

7. Routing Strategy

7.1 Routes

- / → Main portfolio page
- /projects/[id] (optional future expansion)

---

8. State Management

8.1 Approach

- Local state (useState/useReducer)

- No global state library required

---

9. Animation Design

9.1 Libraries

- Framer Motion

9.2 Use Cases

- Page load transitions
- Project card hover effects
- Modal open/close animation

---

10. Performance Requirements

- First Contentful Paint < 2s
- Lighthouse Score > 90
- Lazy loading for images
- Code splitting via Next.js

---

11. SEO Requirements

- Meta tags per page
- Open Graph tags
- Proper heading hierarchy (H1–H3)
- Semantic HTML

---

12. Accessibility

- Keyboard navigation support
- ARIA labels where needed
- Color contrast compliance

---

13. Security Considerations

- No sensitive data exposure
- Sanitize external links
- HTTPS enforced (via hosting)

---

14. Deployment Pipeline

14.1 Steps

1. Push code to GitHub
2. Connect repository to Vercel
3. Auto-build and deploy

14.2 CI/CD

- Automatic deployment on push to main branch

---

15. Logging & Monitoring (Optional)

- Vercel analytics
- Google Analytics (future)

---

16. Demo Handling Strategy

Since projects are local/system-level:

Approach:

- Embed demo video (YouTube/Loom)
- Provide GitHub repo
- Provide setup instructions

---

17. Future Technical Enhancements

- MDX-based blog system
- CMS integration (Sanity/Contentful)
- Backend API for dynamic updates
- GitHub API auto project sync

---

18. Constraints

- No backend initially
- Limited hosting resources
- Heavy projects cannot run live

---

19. Assumptions

- Users have modern browsers
- Stable internet connection
- Recruiters prefer fast-loading sites

---

20. Acceptance Criteria

- Fully responsive UI
- All projects displayed correctly
- Smooth navigation and animations
- Deployment successful and accessible

---

21. Final Technical Insight

This system should prioritize:

- Performance over heavy visuals
- Clarity over complexity
- Structured data over hardcoded UI

The architecture must remain:

- Simple initially
- Scalable for future enhancements
